

BookingHub: Plataforma de Reservas com Alta Concorrência usando PostgreSQL e FastAPI

Carlos Daniel Reis da Silva - 2023010007, Francisco Guilherme Mata Santos - 2024010338, José Kayky Barbosa Coelho - 2023010417

Disciplina de Banco de Dados - Trabalho Final
Projeto BookingHub

Reisdaniel739@gmail.com, 2024010338@unicatolicaquixada.edu.br,
kaykybc01@gmail.com

Repositório do projeto:

<https://github.com/daniel-reis762/bookinghub>

Abstract. This paper presents BookingHub, a backend platform for flight and hotel reservations implemented with FastAPI, PostgreSQL and Docker. The project applies query processing, indexing, explicit SQL transactions, concurrency control and failure recovery. The system uses psycopg2 with direct SQL, implements endpoints for availability, reservations, payments, cancellation and occupancy reports, and includes tests with EXPLAIN ANALYZE, FOR UPDATE locks, SERIALIZABLE isolation, retry logic, logical backup, restore and WAL archiving.

Resumo. Este artigo apresenta o BookingHub, uma plataforma backend de reservas de voos e hotéis implementada com FastAPI, PostgreSQL e Docker. O projeto aplica processamento de consultas, índices, transações SQL explícitas, controle de concorrência e recuperação de falhas. O sistema utiliza psycopg2 com SQL direto, implementa endpoints de disponibilidade, reservas, pagamentos, cancelamento e relatórios de ocupação, além de testes com EXPLAIN ANALYZE, locks FOR UPDATE, isolamento SERIALIZABLE, retry logic, backup lógico, restauração e arquivamento de WAL.

1. Introdução

O BookingHub foi desenvolvido como um backend para gerenciamento de reservas de voos e hotéis. O objetivo foi aplicar conceitos de Banco de Dados em um cenário semelhante ao de uma plataforma real, na qual múltiplos clientes podem consultar disponibilidade, reservar recursos, pagar e cancelar operações simultaneamente.

A arquitetura do sistema utiliza PostgreSQL 16 como SGBD, FastAPI como framework da API REST, psycopg2 como biblioteca de conexão e Docker Compose para padronização do ambiente. A aplicação não utiliza ORM; todas as consultas e comandos transacionais foram escritos diretamente em SQL, conforme exigido no trabalho.

O repositório foi organizado em diretórios separados para a API, scripts de banco, carga de dados, testes, backups, evidências e arquivos de configuração. Essa organização facilita a execução local e a avaliação do projeto.

O código-fonte, scripts de banco de dados, arquivos de teste, documentação e evidências do projeto estão disponíveis publicamente no repositório GitHub do grupo: <https://github.com/daniel-reis762/bookinghub>

2. Modelo de Dados

O modelo de dados representa os principais elementos de uma plataforma de turismo: aeroportos, voos, hotéis, quartos, clientes, reservas e pagamentos. Foram utilizadas chaves primárias, chaves estrangeiras, restrições CHECK e restrições UNIQUE para manter a integridade dos dados.

Tabela	Finalidade	Principais restrições
airports	Cadastro de aeroportos	code UNIQUE e campos obrigatórios
flights	Cadastro de voos	FK para airports, CHECK de horários e assentos
hotels	Cadastro de hotéis	CHECK de estrelas entre 1 e 5
rooms	Quartos de hotéis	UNIQUE(hotel_id, room_number) e CHECK de tipo
customers	Clientes	email UNIQUE e cpf UNIQUE
flight_reservations	Reservas de voo	FK para cliente e voo; UNIQUE(flight_id, seat_number)
hotel_reservations	Reservas de hotel	FK para cliente e quarto; CHECK de datas
payments	Pagamentos	CHECK de tipo, status e método de pagamento

A tabela flights possui os campos total_seats e available_seats. O campo available_seats é atualizado dentro de transações para impedir overbooking. A tabela hotel_reservations armazena intervalos de check-in e check-out, permitindo detectar conflitos de datas em reservas de quartos.

O script seed.py popula o banco com dados realistas usando Faker. Em sua execução validada, o projeto gerou mais de 20 mil registros distribuídos entre as tabelas, permitindo testes de desempenho, consultas e concorrência com volume representativo.

```
>> docker exec -it bookinghub-api-1 python seed.py
Successfully copied 11.4kB (transferred 13.3kB) to bookinghub-api-1:/app/seed.py
Limando banco...
Inserindo aeroportos...
Inserindo clientes...
Inserindo hotéis...
Inserindo quartos...
Inserindo voos...
Inserindo reservas de voo...
Inserindo reservas de hotel...
Inserindo pagamentos...

Seed executado com sucesso!
Aeroportos: 50
Clientes: 2000
Hotéis: 500
Quartos: 5000
Voos: 3000
Reservas de voo: 4000
Reservas de hotel: 3000
Pagamentos: 5204
Total aproximado: 22754
```

Figura 1. Execução do seed.py com mais de 20 mil registros.

```
SELECT 'payments', COUNT(*) FROM payments;
      tabela      | count
-----+-----
airports          |    50
customers         |   2000
hotels            |    500
rooms            |   5000
flights          |   3000
flight_reservations |   4000
hotel_reservations |   3000
payments         |   5271
(8 rows)

bookinghub=#
```

Figura 2. Contagem de registros por tabela após a carga de dados.

3. Implementação da API REST

A API foi desenvolvida com FastAPI e disponibiliza documentação automática no Swagger. Cada endpoint acessa o banco por meio de psycopg2 e executa SQL diretamente. Os endpoints de escrita utilizam transações explícitas, com COMMIT em caso de sucesso e ROLLBACK em situações de erro.

Método	Endpoint	Função
GET	/voos/disponiveis	Lista voos disponíveis
GET	/hoteis/disponiveis	Lista hotéis com quartos livres em um período
GET	/clientes/{cliente_id}/reservas	Lista reservas de voo e hotel do cliente
GET	/relatorios/ocupacao	Gera relatório de ocupação
POST	/reservas/voo	Cria reserva de voo com controle de concorrência
POST	/reservas/hotel	Cria reserva de hotel verificando conflito de datas
POST	/pagamentos	Registra pagamento e confirma reserva
DELETE	/reservas/{reserva_id}	Cancela reserva e estorna disponibilidade
POST	/test/falha-transacao	Simula falha e comprova rollback

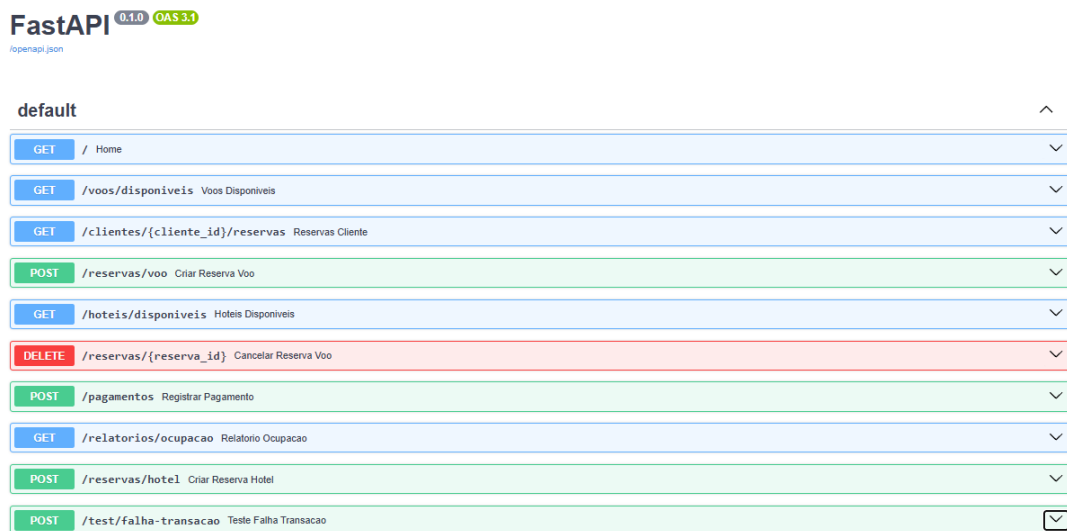


Figura 3. Documentação Swagger com os endpoints implementados.

4. Processamento de Consultas

Foram criados índices B-Tree sobre colunas usadas em filtros, junções e ordenações, como cidades dos aeroportos, origem e destino dos voos, data de partida, status das reservas e histórico por cliente. Também foi criado índice parcial para voos com assentos disponíveis.

```
CREATE INDEX idx_flights_available_departure
ON flights (departure_time)
WHERE available_seats > 0;
```

A análise com EXPLAIN (ANALYZE, BUFFERS) permitiu observar custo estimado, tempo real, uso de cache e operadores de junção. Na consulta C1, o PostgreSQL utilizou Bitmap Index Scan sobre o índice de cidade dos aeroportos, além de Hash Join e Nested Loop para combinar as tabelas. O tempo de execução observado foi baixo para o volume de dados carregado.

```

-----
Sort (cost=122.39..122.51 rows=48 width=48) (actual time=1.433..1.438 rows=33 loops=1)
  Sort Key: f.departure_time
  Sort Method: quicksort Memory: 27kB
  Buffers: shared hit=133
  -> Nested Loop (cost=17.82..126.05 rows=48 width=48) (actual time=0.194..1.387 rows=33 loops=1)
    Buffers: shared hit=133
    -> Hash Join (cost=17.66..189.02 rows=298 width=47) (actual time=0.133..1.160 rows=248 loops=1)
      Hash Cond: (f.destination_airport_id = a2.id)
      Buffers: shared hit=33
      -> Seq Scan on flights f (cost=0.00..83.50 rows=2984 width=46) (actual time=0.068..0.769 rows=2984 loops=1)
        Filter: ((departure_time >= '2026-04-01 00:00:00'::timestamp without time zone) AND (departure_time <= '2026-12-31 00:00:00'::timestamp without time zone) AND
0 (available_seats > 0))
        Rows Removed by Filter: 16
        Buffers: shared hit=31
        -> Hash (cost=16.32..16.32 rows=107 width=9) (actual time=0.056..0.056 rows=4 loops=1)
          Buckets: 1024 Batches: 1 Memory Usage: 9kB
          Buffers: shared hit=2
          -> Bitmap Heap Scan on airports a2 (cost=4.98..16.32 rows=107 width=9) (actual time=0.048..0.049 rows=4 loops=1)
            Recheck Cond: ((city)::text = 'Rio de Janeiro'::text)
            Heap Blocks: exact=1
            Buffers: shared hit=2
            -> Bitmap Index Scan on idx_airports_city (cost=0.00..4.96 rows=107 width=0) (actual time=0.043..0.043 rows=4 loops=1)
              Index Cond: ((city)::text = 'Rio de Janeiro'::text)
              Buffers: shared hit=1
      -> Memoize (cost=0.16..0.20 rows=1 width=9) (actual time=0.001..0.001 rows=0 loops=248)
        Cache Key: f.origin_airport_id
        Cache Mode: logical
        Hits: 198 Misses: 50 Evictions: 0 Overflows: 0 Memory Usage: 4kB
        Buffers: shared hit=100
        -> Index Scan using airports_pkey on airports a1 (cost=0.15..0.19 rows=1 width=9) (actual time=0.001..0.001 rows=0 loops=50)
          Index Cond: (id = f.origin_airport_id)
          Filter: ((city)::text = 'São Paulo'::text)
          Rows Removed by Filter: 1
          Buffers: shared hit=100
Planning:
  Buffers: shared hit=14
Planning Time: 1.069 ms
Execution Time: 1.558 ms
(37 rows)

```

Figura 4. Plano de execução da consulta C1 com junções e uso de índice.

Em teste específico para voos disponíveis ordenados por data, o PostgreSQL utilizou Index Scan sobre o índice parcial. Isso comprova que o otimizador considerou a estrutura criada. Como a consulta retornava grande parte da tabela, o tempo total não necessariamente foi menor que um Seq Scan; esse comportamento é esperado quando muitos registros precisam ser retornados.

```

-----
QUERY PLAN

-----
Index Scan using idx_flights_available_departure on flights (cost=0.28..213.04 rows=2984 width=
24) (actual time=0.419..39.214 rows=2984 loops=1)
  Buffers: shared hit=2882 read=8
  Planning Time: 0.292 ms
  Execution Time: 39.586 ms
(4 rows)

bookinghub=#

```

Figura 5. Index Scan utilizando índice parcial em flights.

Consulta	Objetivo	Operadores observados
C1	Voos disponíveis com filtro	Bitmap Index Scan, Hash Join, Nested Loop
C2	Taxa de ocupação por voo	LEFT JOIN, agregação, GROUP BY
C3	Quartos disponíveis	Subconsulta para evitar conflitos de datas
C4	Histórico do cliente	UNION ALL e LEFT JOIN com payments

5. Transações e Controle de Concorrência

O controle transacional foi implementado diretamente nos endpoints de escrita. A reserva de voo utiliza SELECT ... FOR UPDATE para bloquear a linha do voo antes de verificar disponibilidade e reduzir available_seats. Dessa forma, a verificação e o decremento de vagas ocorrem de forma atômica.

No endpoint de reserva de voo, foi configurado o nível de isolamento SERIALIZABLE e retry logic para tratar falhas de serialização. A política realiza até três tentativas antes de retornar erro final. Esse comportamento atende à necessidade de evitar inconsistências em cenários de concorrência.

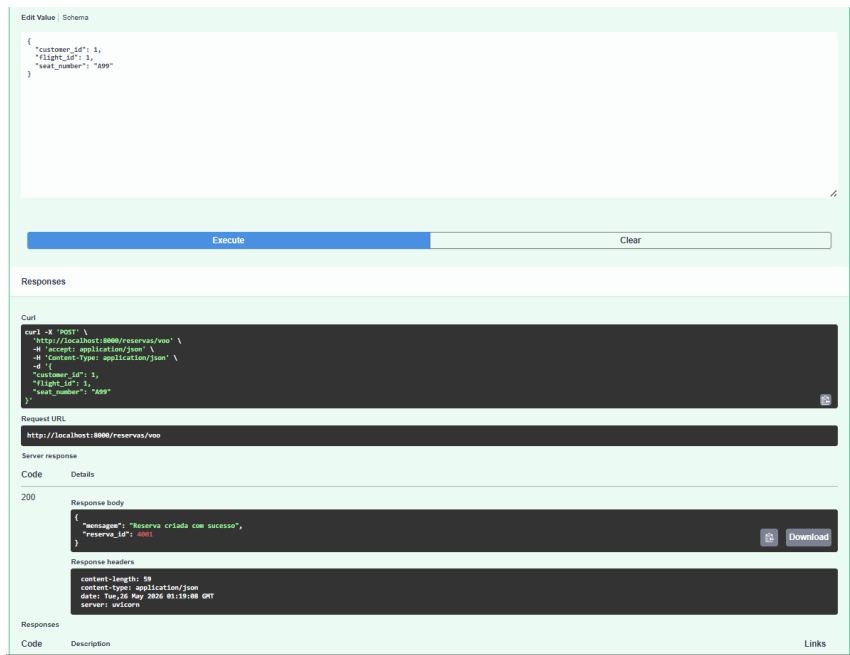


Figura 6. Reserva de voo criada com sucesso.

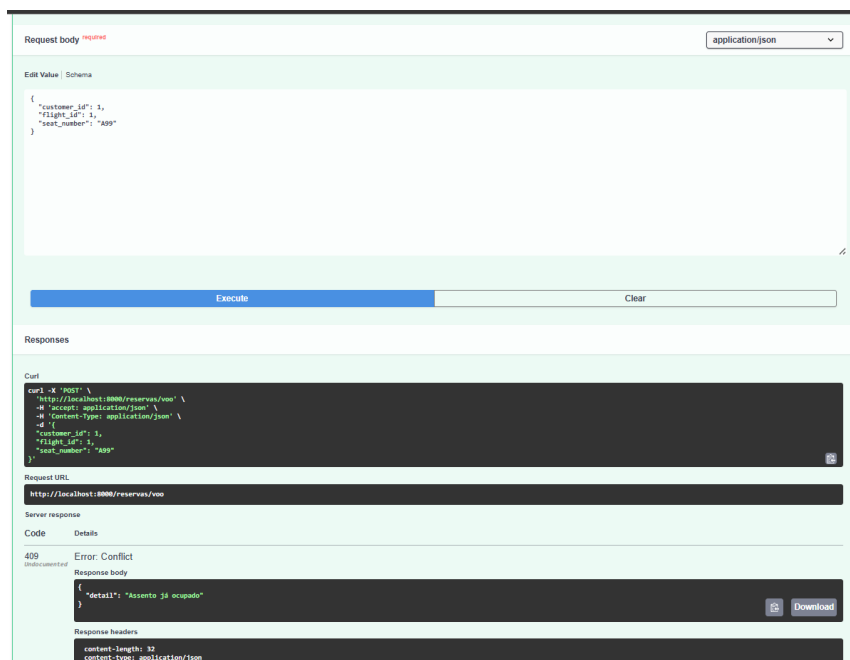


Figura 7. Tentativa duplicada retornando 409 Conflict.

Para hotéis, o endpoint POST /reservas/hotel bloqueia o quarto com FOR UPDATE e verifica se já existe reserva ativa sobreposta ao período solicitado. A regra usa as condições $check_in < novo_check_out$ e $check_out > novo_check_in$. Caso haja conflito, a API retorna 409 Conflict e desfaz a transação.

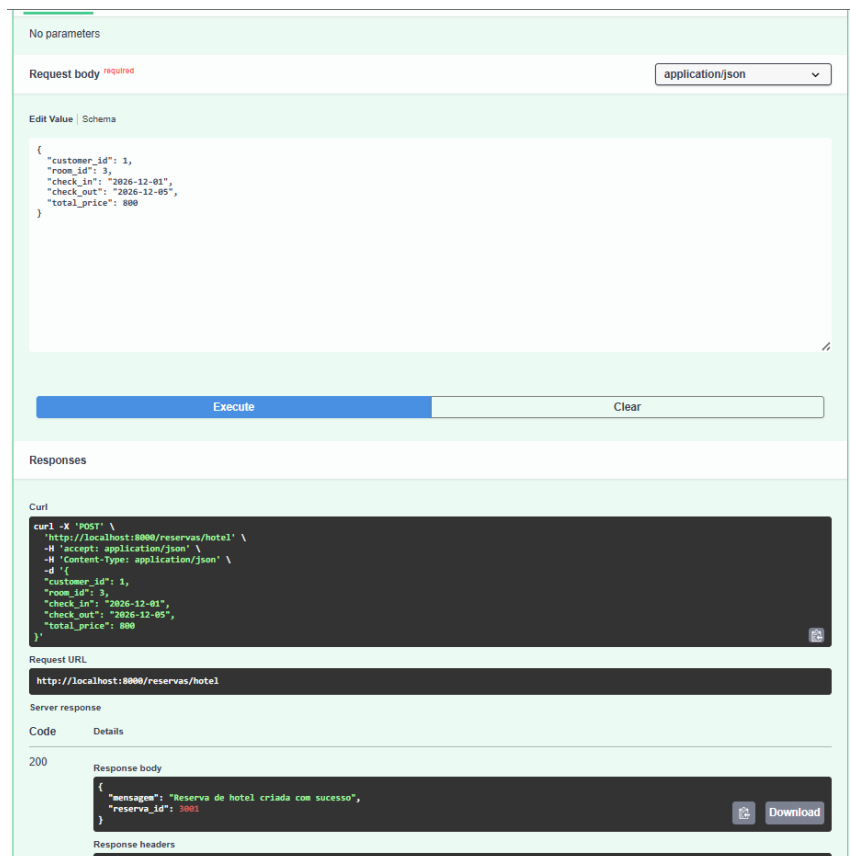


Figura 8. Reserva de hotel criada com sucesso.

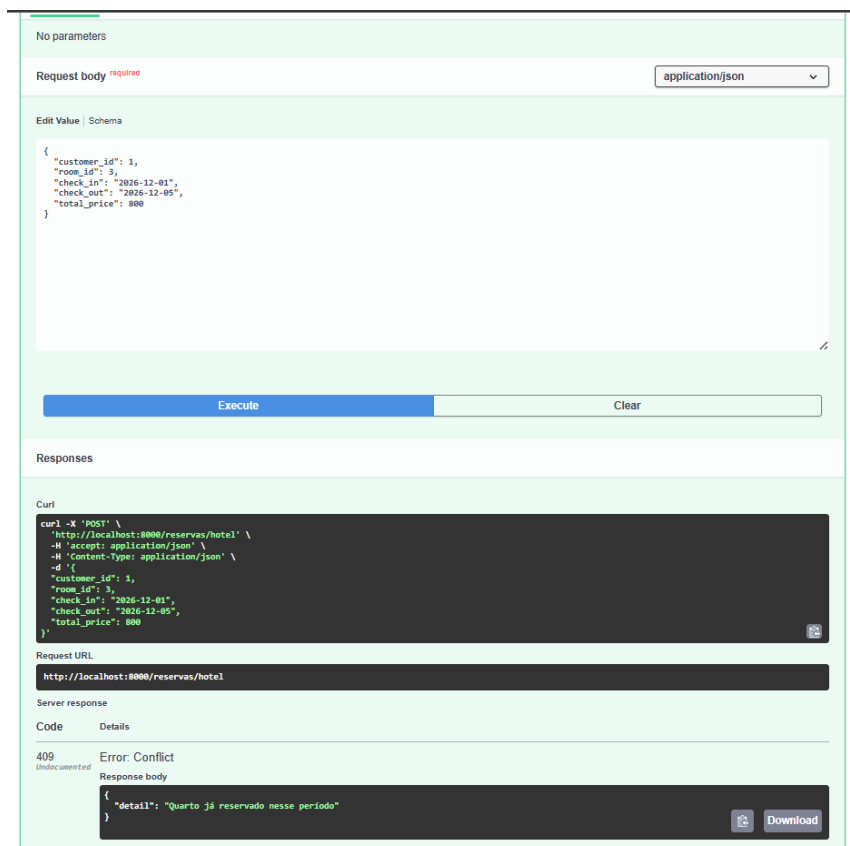


Figura 9. Conflito de datas retornando 409 Conflict.

O arquivo testes/teste_concorrencia.sql demonstra o bloqueio pessimista. O script inicia uma transação `SERIALIZABLE`, seleciona o voo `id = 1` com `FOR UPDATE`, decrementa `available_seats` e mantém o lock com `pg_sleep(15)`. Ao executar outra atualização simultânea no mesmo voo, a segunda sessão fica bloqueada até a liberação da primeira transação.

```
BEGIN;

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

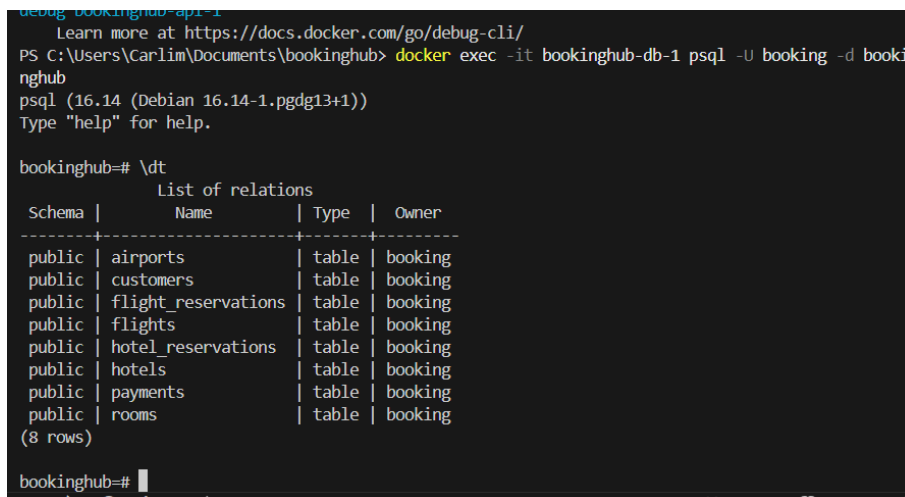
SELECT id, flight_number, available_seats FROM flights WHERE id = 1 FOR
UPDATE;

UPDATE flights SET available_seats = available_seats - 1 WHERE id = 1 AND
available_seats > 0;

SELECT pg_sleep(15);

COMMIT;
```

Também foi criado o endpoint `POST /test/falha-transacao`. Ele insere uma reserva temporária e lança uma exceção antes do `COMMIT`. A consulta posterior por `seat_number = Z99` retornou zero linhas, demonstrando que o `ROLLBACK` impediu a persistência da alteração.



```
debug bookinghub-api-1
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Carlim\Documents\bookinghub> docker exec -it bookinghub-db-1 psql -U booking -d booki
nghub
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

bookinghub=# \dt
          List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | airports       | table | booking
public | customers      | table | booking
public | flight_reservations | table | booking
public | flights        | table | booking
public | hotel_reservations | table | booking
public | hotels         | table | booking
public | payments       | table | booking
public | rooms          | table | booking
(8 rows)

bookinghub=#
```

Figura 10. Resultado da consulta comprovando rollback: zero registros Z99.

6. Recuperação de Falhas

A recuperação de falhas foi implementada por meio de backup lógico, restauração, configuração de WAL e arquivamento de segmentos. O arquivo `docker-compose.yml` monta a pasta `wal_archive` e usa um `postgresql.conf` personalizado para habilitar `archive_mode`.

```
wal_level = replica
archive_mode = on
archive_command = 'cp %p /wal_archive/%f'
archive_timeout = 60
```

```
PS C:\Users\Carlim\Documents\bookinghub> docker exec -it bookinghub-db-1 psql -U booking -d bookinghub -c "SHOW archive_mode; SHOW archive_command; SHOW wal_level;"
archive_mode
-----
on
(1 row)

archive_command
-----
cp %p /wal_archive/%f
(1 row)

wal_level
-----
replica
(1 row)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker
debug bookinghub-db-1
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Carlim\Documents\bookinghub>
```

Figura 11. Validação de archive_mode, archive_command e wal_level.

O WAL (Write-Ahead Log) garante durabilidade porque registra as alterações antes que elas sejam gravadas definitivamente nos arquivos de dados. Em caso de falha, o PostgreSQL usa os registros do WAL para realizar crash recovery e retornar a um estado consistente. O arquivo WAL arquivado no projeto possui nome no formato hexadecimal, por exemplo 00000001000000000000000001, que representa timeline, log e segmento WAL.

```
PS C:\Users\Carlim\Documents\bookinghub>
PS C:\Users\Carlim\Documents\bookinghub> dir wal_archive

Diretório: C:\Users\Carlim\Documents\bookinghub\wal_archive

Mode                LastWriteTime         Length Name
----                -
-a----             27/05/2026    21:31         16777216 000000010000000000000001

PS C:\Users\Carlim\Documents\bookinghub>
```

Figura 12. Arquivo WAL gerado na pasta wal_archive.

O backup lógico foi feito com pg_dump no formato customizado (-Fc), gerado dentro do container para evitar corrupção por redirecionamento binário no PowerShell. A restauração foi testada com pg_restore, recriando as tabelas em banco limpo.

```
docker exec -it bookinghub-db-1 pg_dump -U booking -d bookinghub -Fc -f /tmp/bookinghub.dump

docker cp bookinghub-db-1:/tmp/bookinghub.dump backup/bookinghub.dump

docker exec -it bookinghub-db-1 pg_restore -U booking -d bookinghub --clean --if-exists /tmp/bookinghub.dump
```



```
PS C:\Users\Carlim\Documents\bookinghub> docker exec -it bookinghub-db-1 pg_restore -U booking -d bookinghub --clean --if-exists /tmp/bookinghub.dump

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug bookinghub-db-1
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Carlim\Documents\bookinghub> docker exec -it bookinghub-db-1 psql -U booking -d bookinghub
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

bookinghub=# \dt
               List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | airports       | table | booking
public | customers      | table | booking
public | flight_reservations | table | booking
public | flights        | table | booking
public | hotel_reservations | table | booking
public | hotels         | table | booking
public | payments       | table | booking
public | rooms         | table | booking
(8 rows)

bookinghub=#
```

Figura 13. Tabelas restauradas após pg_restore.

Para o PITR, foram configurados os pré-requisitos de WAL archiving. A recuperação pontual completa exige executar pg_basebackup, inserir registros conhecidos, simular falha e restaurar com restore_command e recovery_target_time. No projeto, o arquivamento WAL foi validado, e os scripts de backup e restauração foram organizados na pasta backup para permitir continuidade do procedimento.

7. Configurações Recomendadas para Produção

Parâmetro	Valor sugerido	Justificativa
max_connections	100-200	Limita conexões simultâneas; excesso degrada desempenho
shared_buffers	25% da RAM	Mantém páginas frequentes em memória e reduz I/O
wal_level	replica	Necessário para PITR e replicação
checkpoint_completion_target	0.9	Distribui o I/O dos checkpoints ao longo do tempo
synchronous_commit	on/off	on favorece durabilidade; off melhora throughput com risco controlado
work_mem	4-64 MB	Afeta operações de sort e hash por consulta

8. Conclusão

O BookingHub permitiu aplicar os principais tópicos da disciplina em um sistema funcional. Foram implementados schema relacional, constraints, índices, carga de dados com mais de 20 mil registros, API REST, consultas otimizadas, transações explícitas, locks, retry logic, rollback, backup lógico, restauração e configuração de WAL.

Os principais desafios encontrados durante o desenvolvimento envolveram a padronização do ambiente com Docker, o controle de concorrência e a execução correta dos procedimentos de backup e recuperação em ambiente Windows. Para resolver os problemas de ambiente e compatibilidade, foi adotado o Docker Compose com containers separados para API e PostgreSQL, garantindo reprodutibilidade da execução em diferentes máquinas. Na geração de dados, foram utilizados scripts em Python com a biblioteca Faker, permitindo popular o banco com milhares de registros consistentes e realistas.

No controle de concorrência, foram utilizados SELECT ... FOR UPDATE e isolamento SERIALIZABLE para evitar overbooking de voos e conflitos de datas em reservas de hotel. Além disso, foi implementada uma estratégia de retry logic com backoff

exponencial para tratar falhas de serialização de transações. Já nos testes de recuperação de falhas, foram configurados mecanismos de WAL archiving, backup lógico com `pg_dump` e restauração com `pg_restore`, permitindo validar a durabilidade e a recuperação do banco após falhas simuladas.

Como melhoria futura, recomenda-se executar a etapa completa de Point-in-Time Recovery (PITR) com `pg_basebackup` e `recovery_target_time`, ampliar os testes automatizados de concorrência com scripts Python e evoluir a API com autenticação, paginação e monitoramento de desempenho.

Referências

PostgreSQL 16 Documentation. Disponível em: <https://www.postgresql.org/docs/16/>.

FastAPI Documentation. Disponível em: <https://fastapi.tiangolo.com/>.

Psycopg2 Documentation. Disponível em: <https://www.psycopg.org/docs/>.

Sociedade Brasileira de Computação. Modelo para publicação de artigos.

Use The Index, Luke! Disponível em: <https://use-the-index-luke.com/>.